

Name:- S. Mahalakshmi

Reg no:- 192524372

Course:- Data Structure

Course code:- CSA0311

1) Compare Heap sort and Quick sort in terms of:-

Basis	Heap Sort	Quick sort
a) Worst-case Guarantee	Worst-case time is $O(n \log n)$	Worst-case time is $O(n^2)$
b) Practical Performance	Usually consistent, but often slower than quick sort because of more comparisons and poorer cache performance	Usually faster in practice due to good cache performance and low overhead
c) Use cases	Useful when a guaranteed $O(n \log n)$ worst-case performance is required	preferred for general-purpose sorting when average-case performance is important.

Time Complexity:-

1) Heap Sort:-

- Best :- $O(n \log n)$
- Average :- $O(n \log n)$
- Worst :- $O(n \log n)$

Quick sort:-

- 1) Best: $O(n \log n)$
- 2) Average: $O(n \log n)$
- 3) Worst: $O(n^2)$

Conclusion:-

Heap sort provides a better worst case guarantee, while quick sort generally provides better practical performance.

2) Compare Radix sort and Counting sort:-

Basis	Radix Sort	Counting Sort
Working mechanism	Sorts numbers digit-by-digit, usually using Bucketing sort as subroutine	Counts the occurrences of each value and uses these counts to place elements
Range of Input values	Suitable for integers / strings with a fixed number of digits or char can handle relatively large range	Best when the range of value k is small compared with the number of elements n .
Efficiency	$O(d(n+k))$, where d = no. of digits k = radix / base	$O(n+k)$

Example:-

Suppose the input is

170, 45, 75, 90, 802, 24, 2, 66

Radix sort:

It sorts according to ones-tens
→ hundreds digits.

Counting sort:

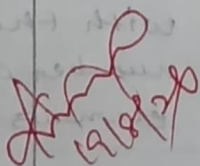
It counts how many times
each value occurs and then reconstructs
the sorted way.

Conclusion:

1) Counting sort is very efficient when
the range of value is small.

2) Radix sort is useful when sorting
large integers or strings with a fixed
number of digits / characters.

3) Radix sort often uses counting sort
internally to sort each digit.


19/8/20